Claude Code: Summary & Key Notes

1. What It Is

- An **agentic coding tool** for the terminal, VS Code, JetBrains, Desktop and web. It reads your code, edits files, runs commands, searches the web and uses Git.
- **Agentic, not conversational:** it gathers context, acts, then verifies the result.
- **Which Claude product to use:**
  - Claude Code for software development.
  - Claude Cowork for research and productivity work.
  - Claude.ai for chat and learning.
  - Claude API for building your own AI apps.

2. Install & Setup

Method	Command
macOS/Linux/WSL	`curl -fsSL https://claude.ai/install.sh | bash`
Windows PowerShell	`irm https://claude.ai/install.ps1 | iex`
npm (Node 18+)	`npm install -g @anthropic-ai/claude-code`
Homebrew / WinGet	`brew install --cask claude-code` / `winget install Anthropic.ClaudeCode`

- Verify with `claude doctor` and `claude --version`. Start by running `claude` in your project folder.
- Never install with `sudo`.
- Sign in with a Claude account or an API key.
- Claude Code can access the launch folder and its subfolders.

3. How It Works

- **Agentic loop:** Gather context → Take action → Verify → repeat. You can steer or interrupt at any point.
- **Models** (switch with `/model` or `claude --model <name>`):
  - Haiku: fast and cheap.
  - Sonnet: recommended default.
  - Opus: complex reasoning.
  - Fable: the highest capability.
  - Always check `/model` for what your account offers.
- **Context window:** everything read, run or typed uses space. It is auto-compacted when full.
- **Checkpoints:**
  - A snapshot is taken before each file edit. Press `Esc` twice to rewind.
  - They restore files only, not side effects such as DB writes, API calls or deployments.
  - They don't replace Git.
- **Sessions:** resume them with `/resume`.

4. Permission Modes (`Shift + Tab` cycles, terminal only)

Mode	Behavior
Manual (default)	Asks before every edit and command.
Edit Automatically	Edits are auto-approved, commands still ask.
Plan Mode	Read-only analysis. Produces a plan and waits for approval. `/plan` works for a single prompt.
Auto Mode	Few prompts. A background safety classifier checks risky actions. It is a research preview for eligible plans.

5. Built-in Tools & Capabilities

- **Tools:** Read, Write, Edit, Grep, Glob, Bash, Web Search, Web Fetch, Code Intelligence.
- **Capabilities:** read and explain code, edit across files, run commands, Git operations, understand a whole repo.

6. Settings & Memory

- **Settings priority:** Managed → User (`~/.claude/settings.json`) → Project (`.claude/settings.json`) → Local (`.claude/settings.local.json`). Permission arrays are merged.
- **CLAUDE.md** holds persistent project instructions: standards, commands, testing, commit style. Keep it concise.
- **Memory hierarchy:**
  - Enterprise policy
  - Project (`./CLAUDE.md` , shared via Git)
  - User (`~/.claude/CLAUDE.md`)
  - Local (`CLAUDE.local.md` , git-ignored)
- **Imports:** use `@path/to/file` inside CLAUDE.md.
- **Auto Memory:** Claude records useful learnings itself. Keep critical rules in CLAUDE.md. Manage memory with `/memory` , and generate a starter file with `/init` .

7. Key Slash Commands

Command	Use
`/init`	Generate CLAUDE.md
`/memory`	Edit memory files
`/clear`	New conversation (memory kept)
`/compact [focus]`	Compress context
`/context`	Show context usage
`/resume`	Resume a session
`/model`	Switch model
`/effort`	Set reasoning effort

8. Extending Claude Code

Feature	Key idea
Skills	A `SKILL.md` holds reusable instructions. Claude invokes it automatically, or you call `/skill-name`. Rule of thumb: if you paste the same instructions twice, make it a skill.
Hooks	Deterministic commands that always fire on events (`PreToolUse`, `PostToolUse`, `Stop`). Exit code `2` blocks an action. Uses: auto-format, protect `.env`, block `rm -rf`.
MCP	Open standard for connecting to external tools (GitHub, Slack, Jira, DBs). `claude mcp add`. Servers can be remote (HTTP) or local (stdio). Servers cost context, so only add ones you trust.
Subagents	Isolated context, restricted tools, return a summary. Built-ins: Explore, Plan, General-purpose. Custom ones live in `.claude/agents/`. They can't spawn other subagents.
Headless	`claude -p "prompt"` for CI/CD and scripts. Use `--allowedTools`, `--output-format json` and `--bare`.
Plugins	Bundle skills, hooks, MCP and agents into one installable package (`.claude-plugin/plugin.json`) and distribute it via a marketplace.

Choosing between them:

- Skills: Claude decides when to use them.
- Hooks: always run.
- Subagents: isolate context.
- MCP: connect external systems.

9. Common Use Cases

- Onboarding to a new codebase
- Bug fixing
- Feature development
- Refactoring and modernization
- Writing tests and TDD
- Documentation
- Git and PR workflows
- Code and security review
- CI/CD automation
- Monorepo refactors
- Non-code knowledge work

10. Best Practices

Do

- Give specific prompts.
- Break work into small steps.
- Review changes.
- Commit often.
- Use Plan Mode for unfamiliar or large work.
- Keep CLAUDE.md lean.
- Run tests.

Don't

- Trust generated code blindly.
- Grant excess permissions.
- Expose secrets.
- Skip tests.
- Cram unrelated tasks into one prompt.
- Use Auto or Bypass mode on untrusted projects.

Security

- Review commands before approving.
- Follow least privilege.
- Keep secrets in environment variables.
- Enforce checks with hooks.

11. Hands-On Modules in This Folder

- **Setup:** install, settings, permission modes, slash commands, CLAUDE.md, memory.
- **Skills and subagents:** skills, subagents, hooks, plugins.
- **Full-stack app build:** Node.js + React, with testing, database, SQL migrations and containerization skills.
- **Vision and MCP:** image-to-code (diagrams, wireframes), Figma MCP, Playwright MCP.
- **Exercises (`Hand'son/`):**
  - Create a CLAUDE.md.
  - Build an app with Plan Mode (Budget Tracker).
  - Create a subagent.
  - Install a skill.
  - Deploy to the web (GitHub + Vercel).
  - Full plan-build-deploy project.

Takeaways

1. Claude Code acts on your project, so you stay in control through permissions and checkpoints.
2. Context is the key resource: use CLAUDE.md, subagents and `/compact`.
3. Plan first, then code, then test, then commit.
4. Use hooks for anything that must always happen.
5. Treat it as a capable teammate: give clear goals and verify the results.